Presentation Script

Campus Shuttle Routing and Capacity Scheduling. Design and Analysis of Algorithms, final review. Total time: about 11 minutes, so you have room to breathe or demo.

Team member	ID
D. Ritheek	25WU0102057
M. Srilakshmi	25WU0102140
B. Deekshan	25WU0102037
P. Priyakaran	25WU0101090
G. Madhan	25WU0102344

The speaker names below are a suggestion so everyone speaks. Swap them however you like, but each person should be able to explain the slide they present and answer questions on it. Press F in the deck for fullscreen and use the arrow keys to move between slides.

1. Title

Suggested speaker: Ritheek | About 30 seconds

Good morning, sir. We are Team 5 from B.Tech AI and ML: Ritheek, Srilakshmi, Deekshan, Priyakaran and Madhan.

Our project for Design and Analysis of Algorithms is **Campus Shuttle Routing and Capacity Scheduling**. We used five families of algorithms in one pipeline to cut the average time a passenger waits for a shuttle across 14 stops on the Woxsen campus.

Cue: Pause, smile, move on.

2. Problem

Suggested speaker: Ritheek | About 45 seconds

The problem is simple. We have 14 stops, from Main Gate to SportX, with real names and GPS coordinates.

If one shuttle simply visits the stops in list order, it zig-zags across campus. The loop takes about 13.4 minutes, and a passenger arriving at random waits half of that on average, which is **6.69 minutes**.

Our goal is to reduce that wait using algorithms from this course.

Cue: Point to the three numbers: 14 stops, 13.37 minutes, 6.69 minutes.

3. Approach

Suggested speaker: Madhan | About 1 minute

First, we model the campus as a weighted graph. Each stop is a vertex, there are 22 roads as edges, and the weight of an edge is the travel time, computed from the distance between the real coordinates at an assumed shuttle speed of 20 kilometres per hour.

On the right is the real layout of the stops. The cyan roads are the minimum spanning tree, which we will explain next.

Then we solve the problem in layers: MST, shortest paths, greedy scheduling, dynamic programming and TSP. Each answer is cross-checked against a second method.

Cue: Point at the map: Main Gate at the top, SportX at the bottom.

4. Graph algorithms

Suggested speaker: Priyakaran | About 1 minute 15 seconds

Three graph algorithms.

Minimum spanning tree. We ran both Prim and Kruskal. They give the same tree with a total of 5.8 minutes, which confirms both are correct. This is the cheapest set of roads connecting every stop.

Shortest path. Dijkstra and Bellman-Ford both find the fastest route from the Bus Stop to Race Arena: 3.3 minutes via Care and Cure, Academic Blocks, Library and Oval. Bellman-Ford also confirms there is no negative cycle.

All pairs. Floyd-Warshall gives every stop-to-stop time, and it matches Dijkstra. The longest shortest trip on campus is Main Gate to Cricket Ground at 4.3 minutes.

Cue: Likely question: Prim vs Kruskal, Dijkstra vs Bellman-Ford. See the viva sheet.

5. Scheduling and allocation

Suggested speaker: Deekshan | About 1 minute

Next, staffing and capacity.

Greedy. We use earliest-finish-first. One driver can cover 8 of the 18 shifts, and repeating the selection shows that 4 drivers can staff all shifts.

Dynamic programming. Assigning passenger groups to vehicles is a 0-1 knapsack: seats are the capacity and passengers times priority is the value. The result fills BUS-1 with 30 of 30 seats, BUS-2 with 19 of 20 and VAN-1 with 9 of 12.

The demand, fleet and shift data here are estimates, and the files say so.

Cue: Be ready to explain why a group cannot be split (that is why it is 0-1 and not fractional).

6. Tour optimisation

Suggested speaker: Srilakshmi | About 1 minute 15 seconds

To decide the order in which a shuttle visits stops, we treat it as the travelling salesman problem.

File order gives a 13.37 minute loop. Nearest neighbour and the MST-based tour both give 10.11 minutes. And Held-Karp, an exact dynamic programming algorithm, gives the true optimum of 9.92 minutes.

So the MST tour is only **1.019 times optimal**, far better than the theoretical guarantee of at most 2 times. Held-Karp is exponential, $O(n^2\ 2^n)$, so it works for 14 stops, but the MST method scales to much larger campuses.

Cue: Likely question: why is the MST tour within 2x of optimal. See the viva sheet.

7. Result

Suggested speaker: Ritheek | About 1 minute

This is the main result. Before, one shuttle in file order: **6.69 minutes** average wait. With one shuttle on the MST tour: 5.05 minutes. With three shuttles running concurrently, each looping one slice of the MST tour: **2.03 minutes**.

That is a **70 percent** reduction in average wait. Routing alone gives about 1.6 minutes of the saving, and adding fleet capacity gives the rest.

Cue: Slow down on the three numbers. This is the slide examiners remember.

8. Fleet sensitivity

Suggested speaker: Madhan | About 45 seconds

We also asked how many shuttles are worth running. Going from two to three shuttles saves 1.88 minutes, but the fourth saves only 0.28 minutes.

So three shuttles is the knee of the curve, and it matches the size of our fleet file.

Cue: Point at the green bar.

9. Complexity

Suggested speaker: Priyakaran | About 45 seconds

This table gives the cost of each algorithm: Prim and Kruskal in O(E log V), Dijkstra in O((V plus E) log V), Floyd-Warshall in $O(V^3)$, greedy scheduling in O(S log S), knapsack in O(nW), and Held-Karp in $O(n^2\ 2^n)$.

We also benchmarked the algorithms on graphs from 50 to 1,600 nodes, and the measured runtimes follow these growth rates.

Cue: Do not read the whole table. Pick two rows.

10. Verification

Suggested speaker: Deekshan | About 1 minute

We tested the project with 202 tests, all passing. The algorithms are checked against each other, and Held-Karp and the knapsack are checked against brute force.

We want to be transparent about the data. The stop names and GPS coordinates are real. The road set, the 20 kilometres per hour speed, the demand, the fleet and the shifts are estimates, and every data file is labelled as an estimate. The algorithms do not change if real measurements replace them, only the exact minutes would.

Cue: Say this clearly. Being upfront about estimates builds trust.

11. Deliverables and future work

Suggested speaker: Srilakshmi | About 45 seconds

The whole project runs with one command, **python main.py**. It runs every algorithm, the benchmarks, and regenerates the dashboard and charts.

For future work, we would survey the real roads and ride times, count real boardings, and add stop dwell time and road-detour factors, then rerun the same code.

Cue: If time allows, offer to open dashboard.html for a live demo.

12. Thank you

Suggested speaker: All | About 15 seconds

Thank you, sir. We would be happy to take your questions.

Cue: Keep the viva sheet open. Whoever knows the algorithm best answers first.

Final tips

• Speak slowly. Do not read the slides word for word; use the script as a guide.

• Look at the examiner, not the screen, when you say the key numbers: 6.69 to 2.03 minutes, 70 percent, 1.019 times optimal.

• If you do not know an answer, say what you do know about the topic, then say you would verify the rest. Do not guess.

• Keep DAA_Viva_Prep open for the question round.